Assignment 2

Supervised Machine Learning Fundamentals

Aesha Gandhi
2026-01-21

```
%pip -q install pandas
%pip -q install scikit-learn
%pip -q install ucimlrepo
%pip -q install nbconvert
```

```
Note: you may need to restart the kernel to use updated packages.
Note: you may need to restart the kernel to use updated packages.
Note: you may need to restart the kernel to use updated packages.
```

```
import numpy
import pandas as pd
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score as accuracy
```

```
import warnings
warnings.filterwarnings("ignore")
```

```
from IPython.display import display, HTML

display(HTML("""
<style>
pre {
    white-space: pre-wrap !important;
    word-wrap: break-word !important;
}
</style>
"""))
```

Exercise 1 - Conceptual Questions on Supervised Learning I

1.1

Flexible would be better in this case becuase flexible models perform well on extremely large data, as they are able to generalize well and when there are less predictors, there is less risk of overfitting to very specific patterns in features.

1.2

Flexible would be worse because with small data and many predictors, the flexible model is more prone to overfitting and will not generalize as well. In this case, inflexible/simpler models tend to generalize better.

1.3

Flexible would be better because when the relationship between predictors and response is nonlinear, flexible models are better at capturing the complex nonlinear patterns in the data, for example, neural networks.

1.4

Flexible would be worse because when the variance of the error terms is high, that means the data is very noisy and the model is overfitting, thus not generalizing well to unseen data.

Exercise 2 - Conceptual Questions on Supervised Learning II

2.1

Regression - This is a regression problem because we are looking at CEO salary which is not categorical.

Inference - This is an inference problem because we are interested in the factors affecting salary, not actually trying to predict values but rather to understand the relationships.

n = 500, p = 3 because we are looking at 500 firms and using 3 predictors: profit, # employees, industry

2.2

Classification - This is a classification problem because we are looking at whether the product is a binary value of success or failure, which are categories.

Prediction - This is prediction because based on a product's features (price, budget, competition price, etc), we want to predict if it will be a success or failure.

n = 20, p = 13, because we have data on 20 products, and use the 13 variables mentioned.

2.3

Regression - This is a regression problem because % change is the value we're interested in predicting, not necessarily any categories, which is continuous.

Prediction - This is prediction because we are looking at predicting a value for % change in US dollar based on the predictors, not necessarily the relationship itself but more about the prediction.

n = 52, p=3, because we have weekly data for an entire year and are using features % change in markets for US, British, and German.

Exercise 3: Classification using KNN

3.1 - Euclidean Distances

```
import numpy as np

X = np.array([[ 0, 3, 0],
              [ 2, 0, 0],
              [ 0, 1, 3],
              [ 0, 1, 2],
              [-1, 0, 1],
              [ 1, 1, 1]])
y = np.array(['r','r','r','b','b','r'])
```

```
test_point = [[0, 0, 0,]]
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))


distances = [euclidean_distance(test_point, point) for point in X]

results = pd.DataFrame(X, columns=['x1', 'x2', 'x3'])
results.insert(0, 'Obs', range(1, len(results) + 1))  # <-- real column
results['distance'] = distances
results['y'] = y

results = results.reset_index(drop=True)

results
```

Out[5]:

	Obs	x1	x2	x3	distance	y
0	1	0	3	0	3.000000	r
1	2	2	0	0	2.000000	r
2	3	0	1	3	3.162278	r
3	4	0	1	2	2.236068	b
4	5	-1	0	1	1.414214	b
5	6	1	1	1	1.732051	r

```
results.sort_values(by='distance')
```

Out[6]:

	Obs	x1	x2	x3	distance	y
4	5	-1	0	1	1.414214	b
5	6	1	1	1	1.732051	r
1	2	2	0	0	2.000000	r
3	4	0	1	2	2.236068	b
0	1	0	3	0	3.000000	r
2	3	0	1	3	3.162278	r

3.2

When K=1, we look at the closest training point to the test point, which is observation 5: [-1, 0, 1] with smallest distance of 1.41 and label b. Thus, we classify the test point with that same label --> blue.

```
results.sort_values(by='distance').head(1)
```

Out[7]:

	Obs	x1	x2	x3	distance	y
4	5	-1	0	1	1.414214	b

3.3

When K=3, we look at the 3 closest training points to the test point (see below code output). Thes are observation 5, 6, and 2 with distances 1.4, 1.7, 2.0, respectively. Then we look at the majority label among these points which are blue, red, red, respectively. Thus we classify our test point as red.

```
results.sort_values(by='distance').head(3)
```

Out[8]:

	Obs	x1	x2	x3	distance	y
4	5	-1	0	1	1.414214	b
5	6	1	1	1	1.732051	r
1	2	2	0	0	2.000000	r

3.4

For a highly nonlinear Bayes decision boundary, smaller K is a better choice because a larger K would be more prone to overfitting while lower K could discover local patterns in the data and be more flexible. There would be lower bias and higher variance.

Exercise 4: Build your own classification algorithm

4.1

```
# Skeleton code for part (a) to write your own kNN classifier

class Knn:
# k-Nearest Neighbor class object for classification training and testing
    def __init__(self):
        self.k = None
        self.X_train = None
        self.y_train = None

    def fit(self, x, y):
        # Save the training data to properties of this class
        self.X_train = x
        self.y_train = y

    def predict(self, x, k):
        y_hat = [] # Variable to store the estimated class label for
        # Calculate the distance from each vector in x to the training data
        for x_vec in x:
            distances = [np.sqrt(np.sum((x_vec - pt ) ** 2)) for pt in self.X_train]
            k_indices = np.argsort(distances)[:k]
            k_nearest_labels = [self.y_train[i] for i in k_indices]

            # majority vote
            pred =  max(k_nearest_labels, key=k_nearest_labels.count)
            y_hat.append(pred)


        # Return the estimated targets
        return y_hat

# Metric of overall classification accuracy
#  (a more general function, sklearn.metrics.accuracy_score, is also available)
def accuracy(y,y_hat):
    nvalues = len(y)
    accuracy = sum(y == y_hat) / nvalues
    return accuracy
```

4.2

```
# load datasets
X_train_low = pd.read_csv('A2_Q4_X_train_low.csv')
X_train_high = pd.read_csv('A2_Q4_X_train_high.csv')

X_test_low = pd.read_csv('A2_Q4_X_test_low.csv')
X_test_high = pd.read_csv('A2_Q4_X_test_high.csv')

y_train_low = pd.read_csv('A2_Q4_y_train_low.csv')
y_train_high = pd.read_csv('A2_Q4_y_train_high.csv')

y_test_low = pd.read_csv('A2_Q4_y_test_low.csv')
y_test_high = pd.read_csv('A2_Q4_y_test_high.csv')


X_train_low  = X_train_low.values.astype(float)
X_test_low   = X_test_low.values.astype(float)

X_train_high = X_train_high.values.astype(float)
X_test_high  = X_test_high.values.astype(float)

y_train_low  = y_train_low.values.ravel()
y_test_low   = y_test_low.values.ravel()

y_train_high = y_train_high.values.ravel()
y_test_high  = y_test_high.values.ravel()
```

4.3

```
# train classifier on low dim dataset and then high dim with k=5
import time

k_vals = [5]

knn_low = Knn()
knn_low.fit(X_train_low, y_train_low)

print("LOW-DIMENSIONAL DATASET")
for k in k_vals:
    t0 = time.perf_counter()
    yhat = knn_low.predict(X_test_low, k)
    t1 = time.perf_counter()
    acc = accuracy(y_test_low, yhat)
    print(f"k={k:>3} | test accuracy = {acc:.4f} | prediction time = {t1 - t0:.6f}
seconds")

print()

knn_high = Knn()
knn_high.fit(X_train_high, y_train_high)

print("HIGH-DIMENSIONAL DATASET")
for k in k_vals:
    t0 = time.perf_counter()
    yhat = knn_high.predict(X_test_high, k)
    t1 = time.perf_counter()
    acc = accuracy(y_test_high, yhat)
    print(f"k={k:>3} | test accuracy = {acc:.4f} | prediction time = {t1 - t0:.6f}
seconds")
```

```
LOW-DIMENSIONAL DATASET
k=  5 | test accuracy = 0.9249 | prediction time = 7.578310 seconds

HIGH-DIMENSIONAL DATASET
k=  5 | test accuracy = 0.9930 | prediction time = 8.323901 seconds
```

4.4

```
# run scikit learn KNN

print("LOW-DIMENSIONAL DATASET (scikit-learn)")

for k in k_vals:
    knn_sklearn = KNeighborsClassifier(n_neighbors=k)
    knn_sklearn.fit(X_train_low, y_train_low)

    t0 = time.perf_counter()
    yhat = knn_sklearn.predict(X_test_low)
    t1 = time.perf_counter()

    acc = accuracy(y_test_low, yhat)
    print(f"k={k:>3} | test accuracy = {acc:.4f} | prediction time = {t1 - t0:.6f}
seconds")

print("\nHIGH-DIMENSIONAL DATASET (scikit-learn)")

for k in k_vals:
    knn_sklearn = KNeighborsClassifier(n_neighbors=k)
    knn_sklearn.fit(X_train_high, y_train_high)

    t0 = time.perf_counter()
    yhat = knn_sklearn.predict(X_test_high)
    t1 = time.perf_counter()

    acc = accuracy(y_test_high, yhat)
    print(f"k={k:>3} | test accuracy = {acc:.4f} | prediction time = {t1 - t0:.6f}
seconds")
```

```
LOW-DIMENSIONAL DATASET (scikit-learn)
k=  5 | test accuracy = 0.9249 | prediction time = 0.005910 seconds

HIGH-DIMENSIONAL DATASET (scikit-learn)
k=  5 | test accuracy = 0.9930 | prediction time = 0.085993 seconds
```

KNN Performance Comparison: Custom Implementation vs. scikit-learn

The custom kNN achieves identical accuracy as the scikit-learn classifier, but since scikit-learn's implementation uses vectorized operations and more optimized methods, its performance is way faster. Our custom kNN takes between 7-9 seconds while scikit learn takes less than 0.05 seconds, which is significantly faster. The custom kNN takes 0.5 seconds longer for the high dimensions meanwhile scikit learn takes much quicker still.

4.5

When the prediction process is slow, it limits how quickly a model can respond to new inputs which may not be ideal for real-time or large scale deployment processes, where we want decisions made frequently and/or with low latency. For example, in fraud detection systems. Slow training is less problematic often because it can be done offline/does not affect the users directly many times.

Exercise 5: Bias-variance tradeoff: exploring the tradeoff with a KNN classifier

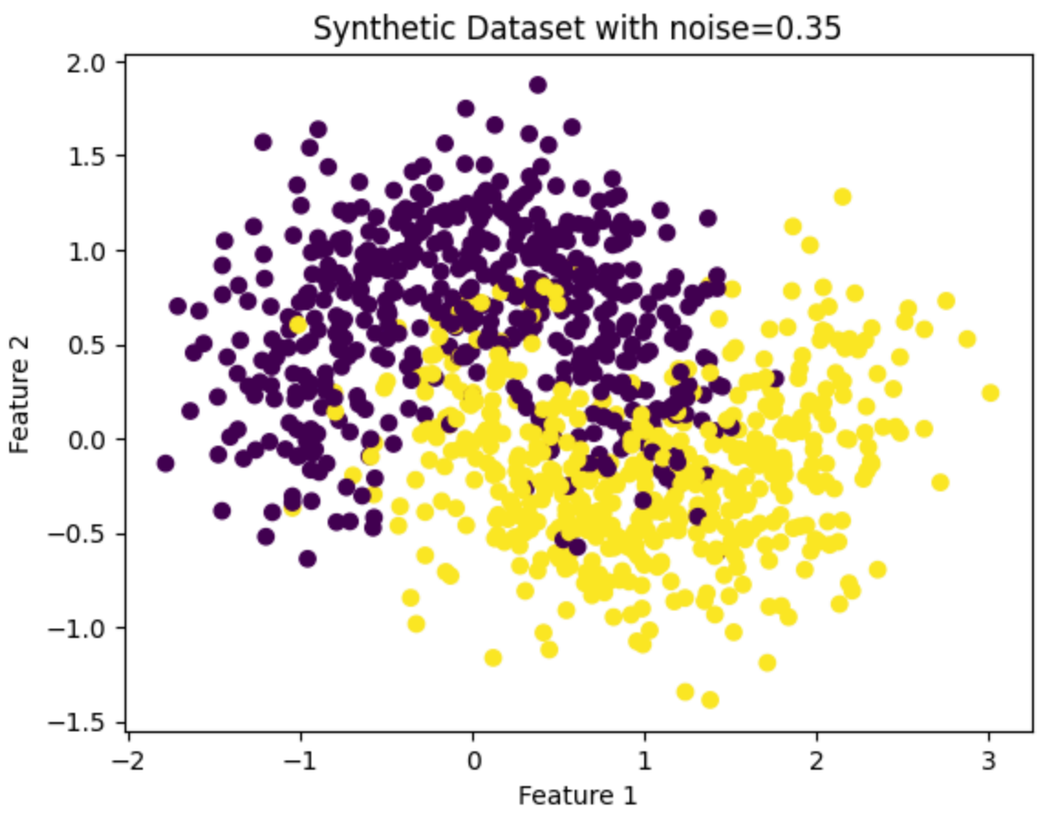
5.1: Create synthetic data

```
# Create a synthetic dataset (with both features and targets). Use the make_moons
module with the parameter noise=0.35 to generate 1000 random samples.
X_train, y_train = make_moons(n_samples=1000, noise=0.35, random_state=42)
X_train.shape, y_train.shape
```

Out[46]: ((1000, 2), (1000,))

5.2: Visualize

```
# visualize the dataset
import matplotlib.pyplot as plt
plt.scatter(X_train[:,0], X_train[:,1], c=y_train)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Synthetic Dataset with noise=0.35')
plt.show()
```

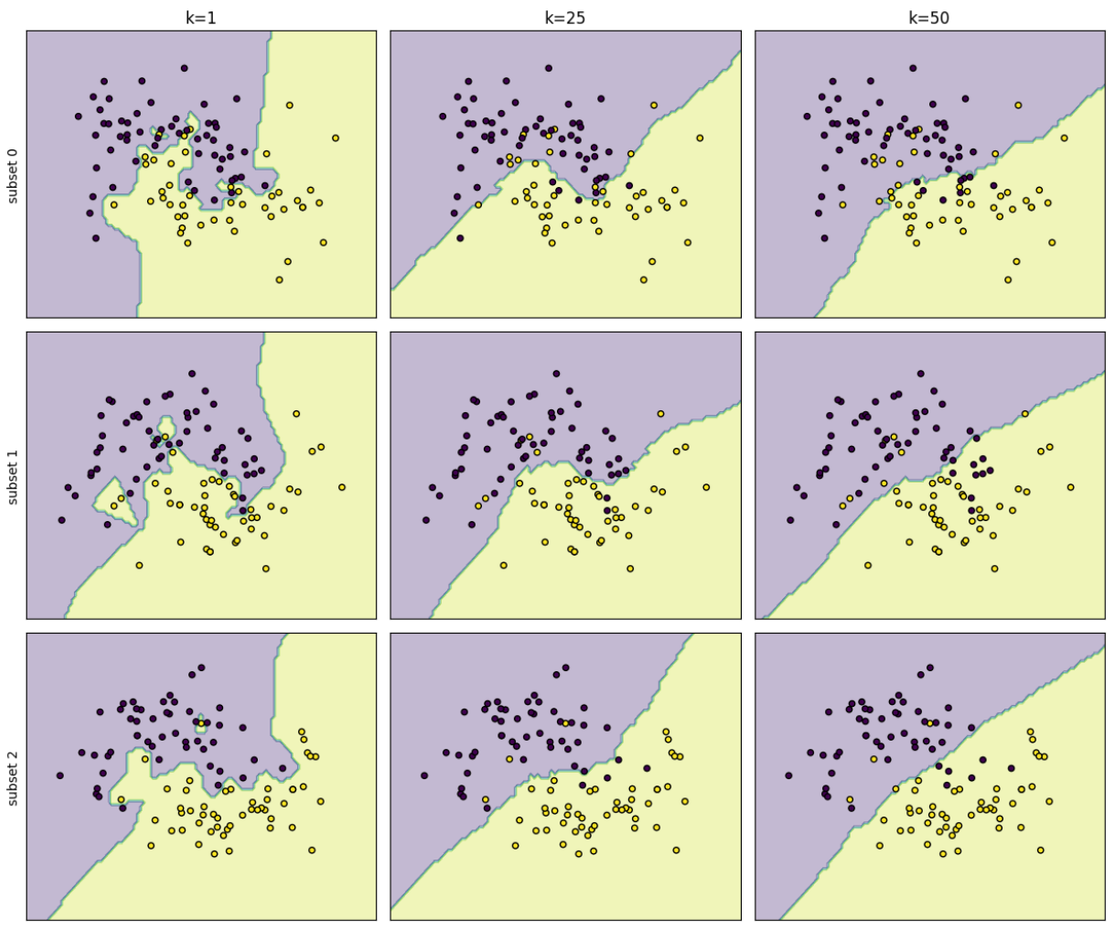


5.3

```
subsets = []
classifiers = []
for i in range(3):
    # get random 100 points out of all data as subset
    indices = np.random.choice(range(X_train.shape[0]), size=100, replace=True)
    X_subset = X_train[indices]
    y_subset = y_train[indices]
    subsets.append((X_subset, y_subset))

    # fit kNN classifiers
    for k in [1, 25, 50]:
        knn = KNeighborsClassifier(n_neighbors=k)
        knn.fit(X_subset, y_subset)
        classifiers.append((knn, k, i))  # store classifier with its k and subset
index
```

5.4

```
# For each combination of dataset and trained classifier plot the decision boundary
(similar in style to Figure 2.15 from Introduction to Statistical Learning). This
should form a 3-by-3 grid. Each column should represent a different value of
# and each row should represent a different dataset.
def plot_grid(subsets, ks=(1, 25, 50), h=0.05):
    allX = np.vstack([Xs for Xs, _ in subsets])
    x_min, x_max = allX[:, 0].min() - 0.5, allX[:, 0].max() + 0.5
    y_min, y_max = allX[:, 1].min() - 0.5, allX[:, 1].max() + 0.5

    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))
    grid = np.c_[xx.ravel(), yy.ravel()]

    fig, axes = plt.subplots(3, 3, figsize=(12, 10), sharex=True, sharey=True)

    for i, (Xs, ys) in enumerate(subsets):
        for j, k in enumerate(ks):
            ax = axes[i, j]
            knn = KNeighborsClassifier(n_neighbors=k).fit(Xs, ys)

            Z = knn.predict(grid).reshape(xx.shape)
            ax.contourf(xx, yy, Z, alpha=0.3)     # decision regions
            ax.scatter(Xs[:, 0], Xs[:, 1], c=ys, edgecolor="k", s=20)

            if i == 0: ax.set_title(f"k={k}")
            if j == 0: ax.set_ylabel(f"subset {i}")
            ax.set_xticks([]); ax.set_yticks([])

    plt.tight_layout()
    plt.show()
plot_grid(subsets, ks=(1, 25, 50))
```



5.5

What do you notice about the difference between the decision boundaries in the rows and the columns in your figure? Which decision boundaries appear to best separate the two classes of data with respect to the training data? Which decision boundaries vary the most as the training data change? Which decision boundaries do you anticipate will generalize best to unseen data and why?

The rows just represent the 3 different random subsets of the data, so we should see similar patterns across rows. There is slight variance across rows which comes from the different datasets. For k=1, the boundaries change more dramtically between rows since tiny changes in the sample can produce different regions. For k=25, there is some variation across rows but the boundaries are stil pretty similar. For k=50, the boundaries are much more similar across rows and we get nearly the same diagonal split each time.

The columns represent the different k values (k = 1, 25, 50) which indicates how many nearest neighbors our KNN algorithm will look at. The plt above shows that for smaller k=1, the two clusters boundary are more defined as almost all of the yellow and black points are not overlapping, which makes sense when only looking at the single closest neighbor. This has a more complex decision boundary, however, this could be not generalizable and overfit when looking at new data.

For the middle column, k = 25, the boundary is a bit more smooth while still fitting most points and adapting to local structures.

For the right column, k = 50, the fit is more general, with very smooth and almost linear decision boundaries, not fully capturing the local structures and missing a few points.

K=1 separates the training data the best, but would likely generalize the worst to unseen data due to its overly complex boundaries that could cause overfitting. K=25 is more likely to generalize best due to a better bias–variance tradeoff.

5.6

The plots from earlier show the bias variance tradeoff by showing that k=1 produces more complex decision boundaries that vary more across the training subsets, showing low bias but high variance. The k=50 plot shows very smooth and stable boundaries across rows, suggesting high bias and low variance which can lead to underfitting. The middle value of k=25 shows a good balance between bias and variance, as it is flexible and stable, and can better generalize likely to unseen data.

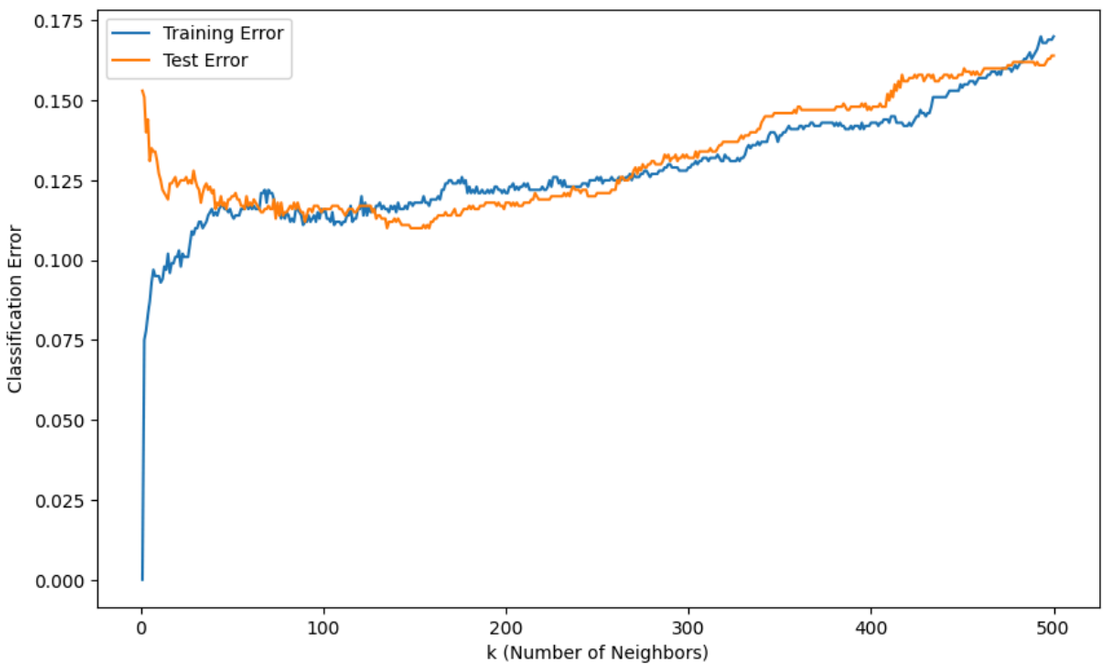
Exercise 6: Bias-variance trade-off II: Quantifying the tradeoff

6.1

```
X_test, y_test = make_moons(n_samples=1000, noise=0.35, random_state=23)
# X_test, y_test
```

6.2

```
train_errors = []
test_errors = []
for k in range(1, 501):
        knn = KNeighborsClassifier(n_neighbors=k)
        # apply to both training and test dataset and plot classification error
        knn.fit(X_train, y_train)
        y_train_pred = knn.predict(X_train)
        y_test_pred = knn.predict(X_test)
        train_error = 1 - accuracy(y_train, y_train_pred)
        test_error = 1 - accuracy(y_test, y_test_pred)

        train_errors.append(train_error)
        test_errors.append(test_error)

        # plot the classification error for both train and test on one plot

ks = range(1, 501)
plt.figure(figsize=(10, 6))
plt.plot(ks, train_errors, label="Training Error")
plt.plot(ks, test_errors, label="Test Error")
plt.xlabel("k (Number of Neighbors)")
plt.ylabel("Classification Error")
plt.legend()
plt.show()
```



6.3

As k increases, the training error increases while the test error initially decreases for lower k and then increases, forming a U shaped curve. This shows the bias variance tradeoff, as smaller k values overfit and larger values underfit.

6.4

Smaller values of k such as k=1 represent high bias since the model is very flexible and sensitive to noise. Larger values of k such as k>300 represent high variance since the model boundaries are too smooth and fail to capture the local structure patterns in data.

6.5

To find optimal k, we want to see where test error is minimized and we can balance bias and variance. In this plot, this is around k=100 to 150, where there is a dip in the classification errors. Through computation in python by seeing when the test error is minimized, the best k is at k = 135 with test error of 0.1099.

6.6

In other models, other hyperparameters control the regularization or model flexibility. For linear regression, there is degree of polynomial features and regularization strength. For decision tree, there is max depth and min samples per leaf. For neural nets, there are number of layers, number of hidden units, regularization.

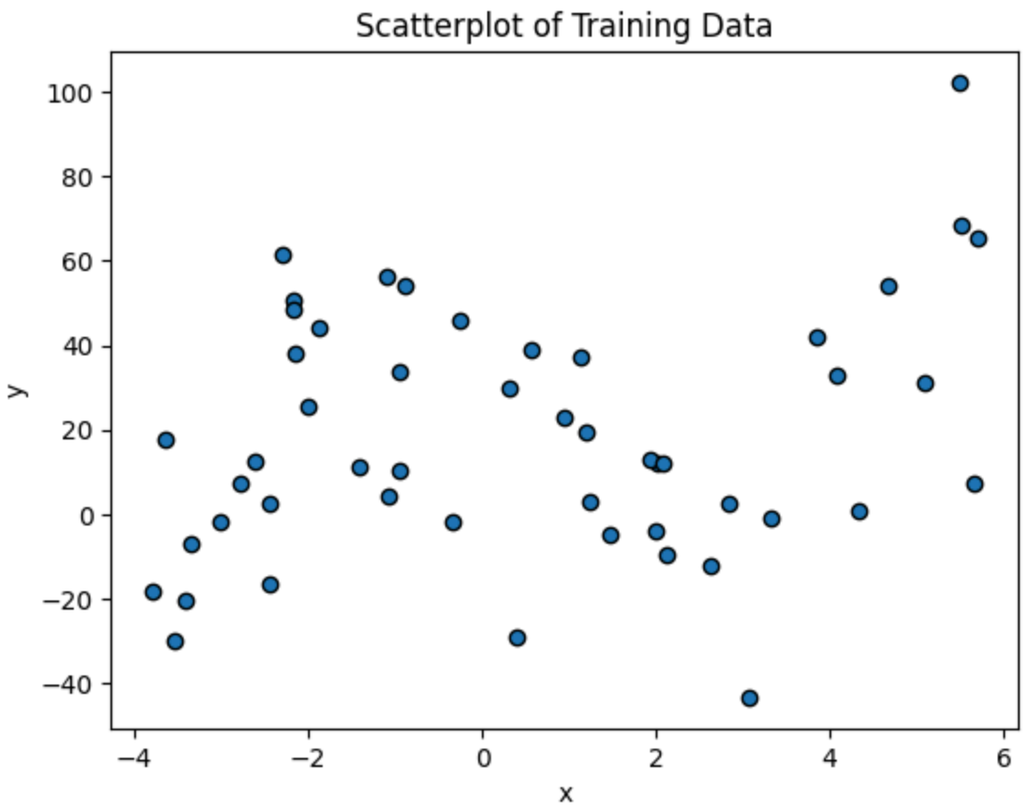
```
ks = np.arange(1, 501)
best_idx = np.argmin(test_errors)
best_k = ks[best_idx]
best_test_error = test_errors[best_idx]

best_k, best_test_error
```

```
Out[20]:  (np.int64(135), np.float64(0.10999999999999999))
```

Exercise 7: Linear regression and nonlinear transformations

7.1

```
q7_train = pd.read_csv('./A2_Q7_train.csv')
q7_test = pd.read_csv('./A2_Q7_test.csv')

# create scatterplot of data
plt.scatter(q7_train.x, q7_train.y, edgecolor='k')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Scatterplot of Training Data')
plt.show()
```



7.2

```
# linear regression model for training
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
lin_reg = LinearRegression()
X_train = q7_train[['x']]
y_train_values = q7_train['y'].values
lin_reg.fit(X_train, y_train_values)

# calculate R squared for training data
y_train_pred = lin_reg.predict(X_train)

r_squared_train = r2_score(y_train_values, y_train_pred)
mse_train = mean_squared_error(y_train_values, y_train_pred)


# calculate mean squared error for training data

print(f"Training Data: R-squared = {r_squared_train}, MSE = {mse_train}")

# . Also provide the equation representing the estimated model (e.g. , but with the
estimated coefficients inserted. from training fit
slope = lin_reg.coef_[0]
intercept = lin_reg.intercept_

print(f"Model: y = {slope} * x + {intercept}")
```

```
Training Data: R-squared = 0.06486123304769698, MSE = 791.4167471701106
Model: y = 2.5907282580761315 * x + 17.204928179405222
```

$\hat{y} = 2.5907\backslash.x + 17.2049$

7.3

```
# build model as y = a0 x + a1 x^2
# add x squared as a feature
x_train = np.asarray(q7_train[['x']]).reshape(-1, 1)
y_train = np.asarray( q7_train['y']).reshape(-1, )

Z_train = np.column_stack([
    x_train.ravel(),
    (x_train.ravel() ** 2)
])

model = LinearRegression()
model.fit(Z_train, y_train)

a0 = model.intercept_
a1, a2 = model.coef_

yhat_train = model.predict(Z_train)

r2_train = r2_score(y_train, yhat_train)
mse_train = mean_squared_error(y_train, yhat_train)


print(f"Training Data: R squared  = {r2_train:.4f}")
print(f"MSE  = {mse_train:.4f}")

print(f"Model: y_hat = {a0:.4f} + ({a1:.4f})*x + ({a2:.4f})*x^2")
```

```
Training Data: R squared  = 0.0853
MSE  = 774.1273
Model: y_hat = 12.9445 + (1.6056)*x + (0.5618)*x^2
```

Model:

$\hat{y} = 12.9445 + 1.6056\,x + 0.5618\,x^2$ `

7.4

```
# visualize model fit to training data, plot iriginal data and both curves
import numpy as np
import matplotlib.pyplot as plt

x = q7_train["x"].values
y = q7_train["y"].values

x_grid = np.linspace(x.min(), x.max(), 400).reshape(-1, 1)

y_lin_curve = lin_reg.predict(x_grid)

Z_grid = np.column_stack([x_grid.ravel(), (x_grid.ravel() ** 2)])
y_quad_curve = model.predict(Z_grid)

plt.figure(figsize=(8, 5))
plt.scatter(x, y, label="Training data")
plt.plot(x_grid, y_lin_curve, label="Linear model (b)")
plt.plot(x_grid, y_quad_curve, label="Quadratic model (c)")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Training Data with Model Fits")
plt.legend()
plt.show()

print(f"Linear model (b): y_hat = {intercept:.4f} + ({slope:.4f})*x")
print(f"Quadratic model (c): y_hat = {a0:.4f} + ({a1:.4f})*x + ({a2:.4f})*x^2")
```


```
Linear model (b): y_hat = 17.2049 + (2.5907)*x
Quadratic model (c): y_hat = 12.9445 + (1.6056)*x + (0.5618)*x^2
```

7.5

```
X_test = q7_test[['x']].values
y_test = q7_test['y'].values


y_test_pred_lin = lin_reg.predict(X_test)

r2_test_lin = r2_score(y_test, y_test_pred_lin)
mse_test_lin = mean_squared_error(y_test, y_test_pred_lin)

print("Linear model (b) — Test Data:")
print(f"R^2 = {r2_test_lin:.4f}")
print(f"MSE = {mse_test_lin:.4f}")
print()


x_test = np.asarray(X_test).reshape(-1, 1)

Z_test = np.column_stack([
    x_test.ravel(),
    x_test.ravel() ** 2
])

y_test_pred_quad = model.predict(Z_test)

r2_test_quad = r2_score(y_test, y_test_pred_quad)
mse_test_quad = mean_squared_error(y_test, y_test_pred_quad)

print("Quadratic model (c) — Test Data:")
print(f"R^2 = {r2_test_quad:.4f}")
print(f"MSE = {mse_test_quad:.4f}")
```

```
Linear model (b) — Test Data:
R^2 = -0.1329
MSE = 1116.6632

Quadratic model (c) — Test Data:
R^2 = -0.1203
MSE = 1104.2014
```

7.6: Model Performance Summary

Dataset	Model	R^2	MSE
Train	Linear	0.0649	791.4167
Train	Quadratic	0.0853	774.1273
Test	Linear	-0.1329	1116.6632
Test	Quadratic	-0.1203	1104.2014

On the training data, the linear model has lower R squared than the transformed (quadratic) model and the transformed model has slightly lower Mean Squared Error, thus the transformed/quadratic model performs better than the linear model on the training data.

On the test data, the quadratic model has a slightly lower negative R squared and lower Mean Squared Error, thus performing better than the linear model on the test set.

These results make sense because the quadratic model is more flexible and can better fit the patterns in the training data. However, on the test set we see both models generalize poorly overal, so even while the quadratic model captures some nonlinearity, both models do not explain the variation in the new unseen data well. Both models already had low training R squared to begin with, and the negative test R squared means the relationship is very weak/noisy.

7.7

If the test data was very different from the training dataset, the models performance would decrease because during training, the model learned patterns specific to the training distribution. Then the generalization accuracy would decrease with higher MSE or lower/more negative R squared. This is because the model is not able to interpolate since the inputs in the test data would not look similar to the training data or follow similar distributions. Then, the model is unable to adapt and make confident predictions since it was not trained on data that looks like the test at all.

Bonus

Implement a KNN with good accuracy latency tradeoff using Census Income dataset.

Step 1: Load Data

```
from ucimlrepo import fetch_ucirepo

# fetch dataset
adult = fetch_ucirepo(id=2)

# data (as pandas dataframes)
X = adult.data.features
y = adult.data.targets

# metadata
print(adult.metadata)

# variable information
print(adult.variables)
```

{'uci_id': 2, 'name': 'Adult', 'repository_url': 'https://archive.ics.uci.edu/dataset/2/adult', 'data_url': 'https://archive.ics.uci.edu/static/public/2/data.csv', 'abstract': 'Predict whether annual income of an individual exceeds $50K/yr based on census data. Also known as "Census Income" dataset. ', 'area': 'Social Science', 'tasks': ['Classification'], 'characteristics': ['Multivariate'], 'num_instances': 48842, 'num_features': 14, 'feature_types': ['Categorical', 'Integer'], 'demographics': ['Age', 'Income', 'Education Level', 'Other', 'Race', 'Sex'], 'target_col': ['income'], 'index_col': None, 'has_missing_values': 'yes', 'missing_values_symbol': 'NaN', 'year_of_dataset_creation': 1996, 'last_updated': 'Tue Sep 24 2024', 'dataset_doi': '10.24432/C5XW20', 'creators': ['Barry Becker', 'Ronny Kohavi'], 'intro_paper': None, 'additional_info': {'summary': "Extraction was done by Barry Becker from the 1994 Census database. A set of reasonably clean records was extracted using the following conditions: ((AAGE>16) && (AGI>100) && (AFNLWGT>1)&& (HRSWK>0))\n\nPrediction task is to determine whether a person's income is over $50,000 a year.\n", 'purpose': None, 'funded_by': None, 'instances_represent': None, 'recommended_data_splits': None, 'sensitive_data': None, 'preprocessing_description': None, 'variable_info': 'Listing of attributes:\r\n\r\n>50K, <=50K.\r\n\r\nage: continuous.\r\nworkclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.\r\nfnlwgt: continuous.\r\neducation: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.\r\neducation-num: continuous.\r\nmarital-status: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse.\r\noccupation: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.\r\nrelationship: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.\r\nrace: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.\r\nsex: Female, Male.\r\ncapital-gain: continuous.\r\ncapital-loss: continuous.\r\nhours-per-week: continuous.\r\nnative-country: United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinadad&Tobago, Peru, Hong, Holand-Netherlands.', 'citation': None}}

	name	role	type	demographic	description	units	missing_values
0	age	Feature	Integer	Age	N/A	None	no
1	workclass	Feature	Categorical	Income	Private, Self-emp-not-inc, Self-emp-inc, Feder...	None	yes
2	fnlwgt	Feature	Integer	None	None	None	no
3	education	Feature	Categorical	Education Level	Bachelors, Some-college, 11th, HS-grad, Prof-...	None	no
4	education-num	Feature	Integer	Education Level	None	None	no
5	marital-status	Feature	Categorical	Other	Married-civ-spouse, Divorced, Never-married, S...	None	no
6	occupation	Feature	Categorical	Other	Tech-support, Craft-repair, Other-service, Sal...	None	yes
7	relationship	Feature	Categorical	Other	Wife, Own-child, Husband, Not-in-family, Other...	None	no
8	race	Feature	Categorical	Race	White, Asian-Pac-Islander, Amer-Indian-Eskimo,...	None	no
9	sex	Feature	Binary	Sex	Female, Male.	None	no
10	capital-gain	Feature	Integer	None	None	None	no
11	capital-loss	Feature	Integer	None	None	None	no
12	hours-per-week	Feature	Integer	None	None	None	no
13	native-country	Feature	Categorical	Other	United-States, Cambodia, England, Puerto-Rico,...	None	yes
14	income	Target	Binary	Income	>50K, <=50K.	None	no

```
df = pd.concat([X, y], axis=1)
df.head()
```

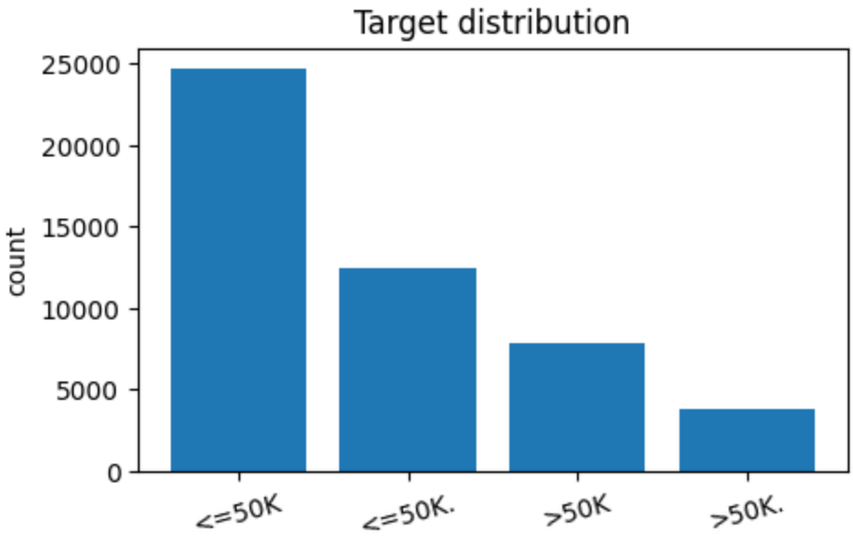
Out[27]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife

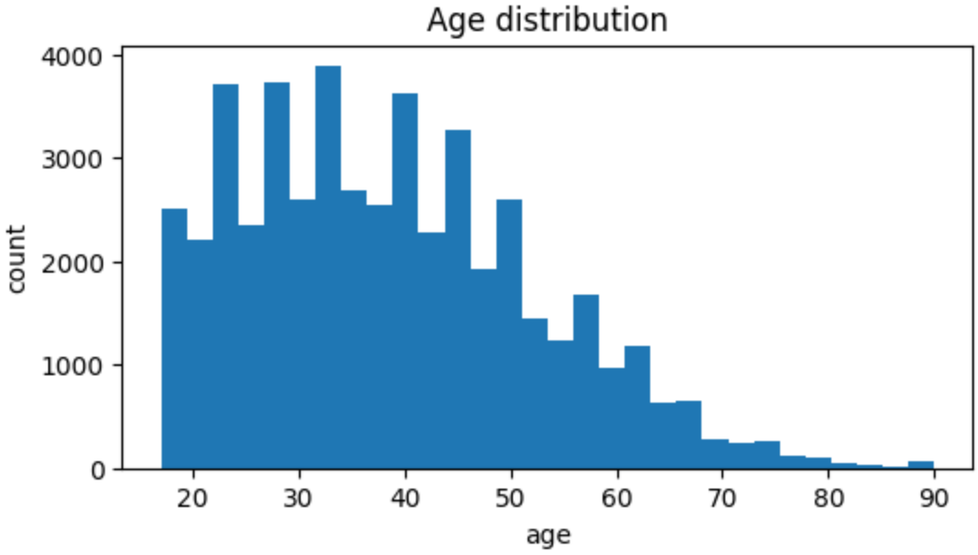
Step 2: EDA

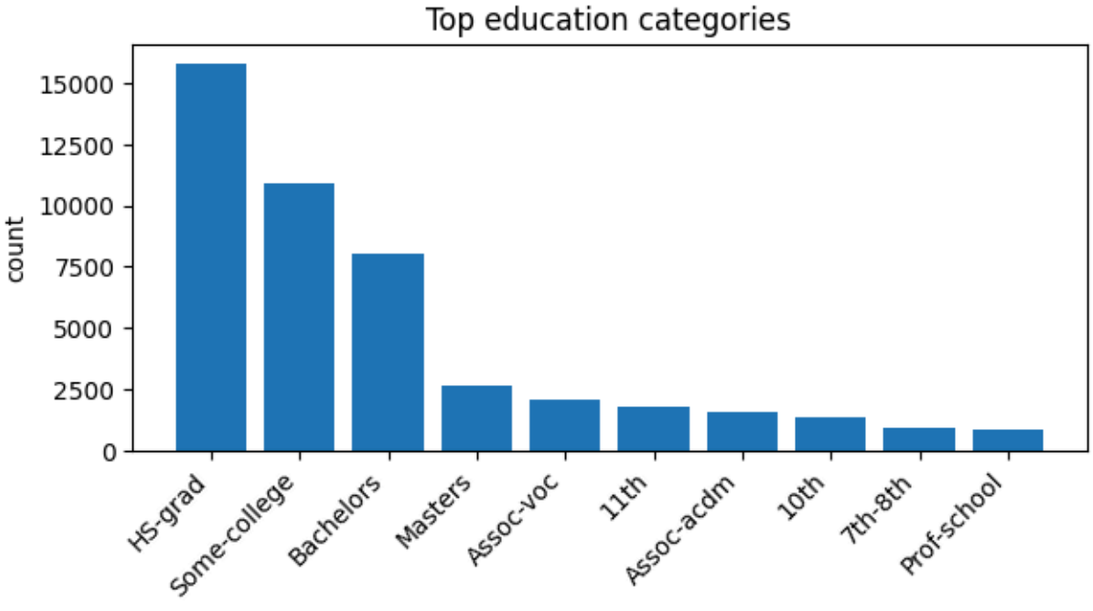
```
print(df.shape)
print(df["income"].value_counts(dropna=False))

# view balance of income column
counts = df["income"].value_counts()
plt.figure(figsize=(5,3))
plt.bar(counts.index.astype(str), counts.values)
plt.title("Target distribution")
plt.ylabel("count")
plt.xticks(rotation=15)
plt.show()

# see missing values of top 10 columns
missing = df.isna().mean().sort_values(ascending=False)
print("Top missing columns:\n", missing.head(10))

# distribution of age
plt.figure(figsize=(6,3))
plt.hist(df["age"].dropna(), bins=30)
plt.title("Age distribution")
plt.xlabel("age")
plt.ylabel("count")
plt.show()

# distribution of agw
edu_counts = df["education"].value_counts().head(10)
plt.figure(figsize=(7,3))
plt.bar(edu_counts.index.astype(str), edu_counts.values)
plt.title("Top education categories")
plt.xticks(rotation=45, ha="right")
plt.ylabel("count")
plt.show()
```

```
(48842, 15)
income
<=50K     24720
<=50K.    12435
>50K       7841
>50K.      3846
Name: count, dtype: int64
```




```
Top missing columns:
 occupation        0.019778
workclass          0.019717
native-country     0.005610
fnlwgt             0.000000
education          0.000000
education-num      0.000000
age                0.000000
marital-status     0.000000
relationship       0.000000
sex                0.000000
dtype: float64
```





Step 3: Preprocessing

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.decomposition import TruncatedSVD

# Separate X/y
y = (df["income"].astype(str).str.contains(">50K")).astype(int)
X = df.drop(columns=["income"])

# Identify column types
numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = [c for c in X.columns if c not in numeric_cols]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric_cols),
    ("cat", categorical_pipe, categorical_cols)
], remainder="drop")
```

Step 4: Evaluation and Baseline

```
import time
from sklearn.metrics import accuracy_score, f1_score

def evaluate_model(model, X_train, y_train, X_test, y_test, timing_reps=5):
    # fit
    model.fit(X_train, y_train)

    # predict metrics
    yhat = model.predict(X_test)
    acc = accuracy_score(y_test, yhat)
    f1 = f1_score(y_test, yhat)

    # latency: average ms per prediction
    times = []
    for _ in range(timing_reps):
        t0 = time.perf_counter()
        _ = model.predict(X_test)
        t1 = time.perf_counter()
        times.append(t1 - t0)

    avg_time = float(np.mean(times))
    ms_per_pred = 1000.0 * avg_time / len(X_test)

    return {"accuracy": acc, "f1": f1, "ms_per_pred": ms_per_pred}
```

```
from sklearn.neighbors import KNeighborsClassifier

baseline_knn = Pipeline([
    ("prep", preprocess),
    ("knn", KNeighborsClassifier(n_neighbors=25))
])

baseline_results = evaluate_model(baseline_knn, X_train, y_train, X_test, y_test)
baseline_results
```

```
Out[31]: {'accuracy': 0.84612234870199,
          'f1': 0.6522302424578937,
          'ms_per_pred': 1.0809216135444502}
```

Step 5: Experiment

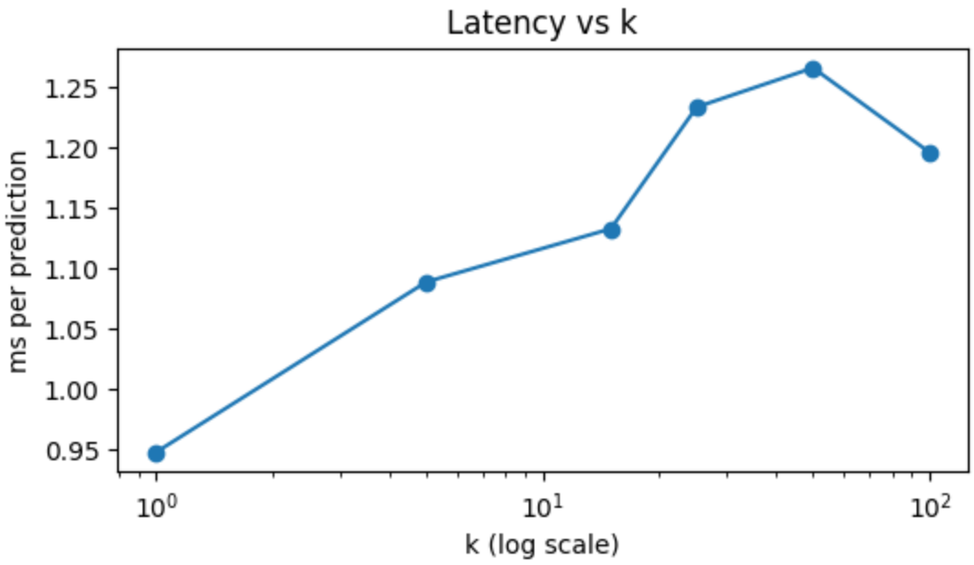
Vary Values of K

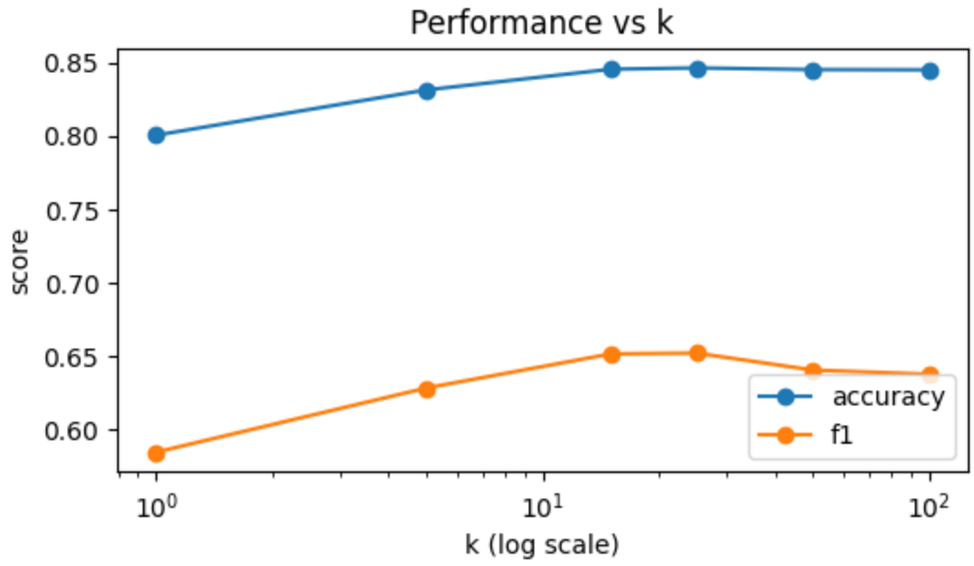
```
k_values = [1, 5, 15, 25, 50, 100]
rows = []

for k in k_values:
    model = Pipeline([
        ("prep", preprocess),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    out = evaluate_model(model, X_train, y_train, X_test, y_test)
    out.update({"setting": "vary_k", "k": k})
    rows.append(out)

res_k = pd.DataFrame(rows)
res_k
```

Out[32]:

	accuracy	f1	ms_per_pred	setting	k
0	0.800344	0.584809	0.947976	vary_k	1
1	0.831136	0.628468	1.088804	vary_k	5
2	0.845222	0.651677	1.132744	vary_k	15
3	0.846122	0.652230	1.233352	vary_k	25
4	0.844976	0.640729	1.265776	vary_k	50
5	0.844812	0.637875	1.195947	vary_k	100

```
plt.figure(figsize=(6,3))
plt.plot(res_k["k"], res_k["ms_per_pred"], marker="o")
plt.xscale("log")
plt.xlabel("k (log scale)")
plt.ylabel("ms per prediction")
plt.title("Latency vs k")
plt.show()

plt.figure(figsize=(6,3))
plt.plot(res_k["k"], res_k["accuracy"], marker="o", label="accuracy")
plt.plot(res_k["k"], res_k["f1"], marker="o", label="f1")
plt.xscale("log")
plt.xlabel("k (log scale)")
plt.ylabel("score")
plt.title("Performance vs k")
plt.legend()
plt.show()
```





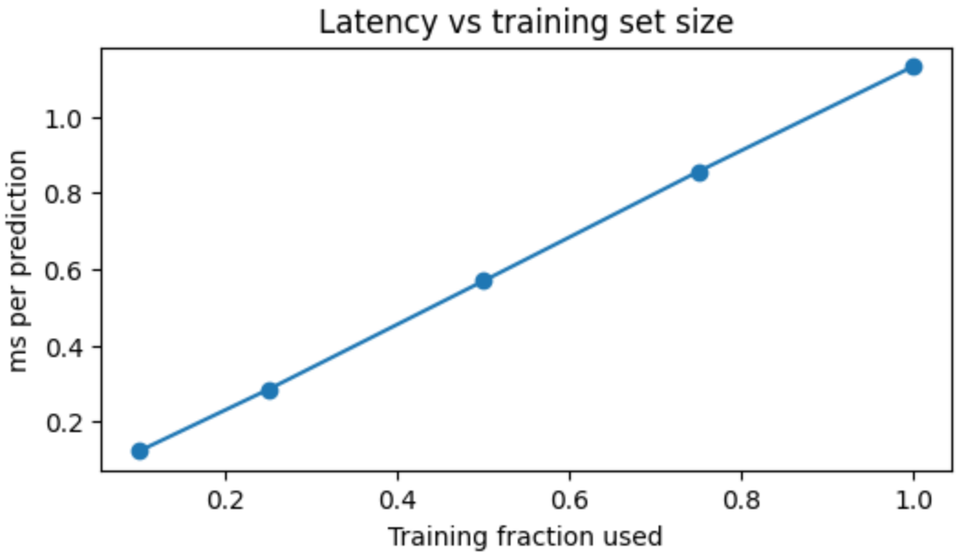
Vary training size (subsample training set)

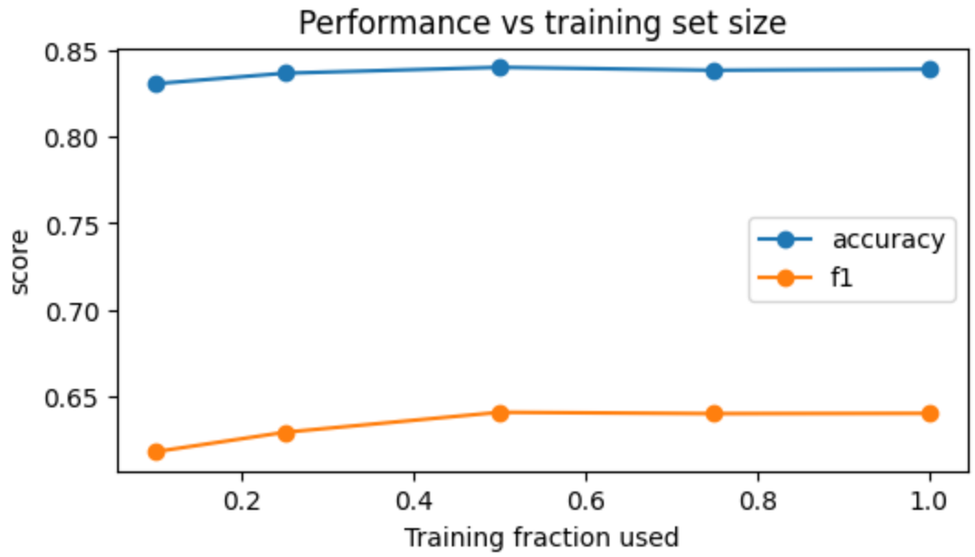
```
from sklearn.utils import resample

fractions = [0.1, 0.25, 0.5, 0.75, 1.0]
k_fixed = 25
rows = []

for frac in fractions:
    X_sub, y_sub = resample(X_train, y_train, n_samples=int(frac * len(X_train)),
random_state=42, stratify=y_train)

    model = Pipeline([
        ("prep", preprocess),
        ("knn", KNeighborsClassifier(n_neighbors=k_fixed))
    ])

    out = evaluate_model(model, X_sub, y_sub, X_test, y_test)
    out.update({"setting": "vary_train_frac", "train_frac": frac, "k": k_fixed})
    rows.append(out)

res_frac = pd.DataFrame(rows)
res_frac
```

Out[34]:

	accuracy	f1	ms_per_pred	setting	train_frac	k
0	0.830972	0.617636	0.123811	vary_train_frac	0.10	25
1	0.837032	0.628870	0.285053	vary_train_frac	0.25	25
2	0.840472	0.640458	0.569065	vary_train_frac	0.50	25
3	0.838752	0.639707	0.855794	vary_train_frac	0.75	25
4	0.839407	0.639853	1.131323	vary_train_frac	1.00	25

```
plt.figure(figsize=(6,3))
plt.plot(res_frac["train_frac"], res_frac["ms_per_pred"], marker="o")
plt.xlabel("Training fraction used")
plt.ylabel("ms per prediction")
plt.title("Latency vs training set size")
plt.show()

plt.figure(figsize=(6,3))
plt.plot(res_frac["train_frac"], res_frac["accuracy"], marker="o", label="accuracy")
plt.plot(res_frac["train_frac"], res_frac["f1"], marker="o", label="f1")
plt.xlabel("Training fraction used")
plt.ylabel("score")
plt.title("Performance vs training set size")
plt.legend()
plt.show()
```





Vary Number of Features

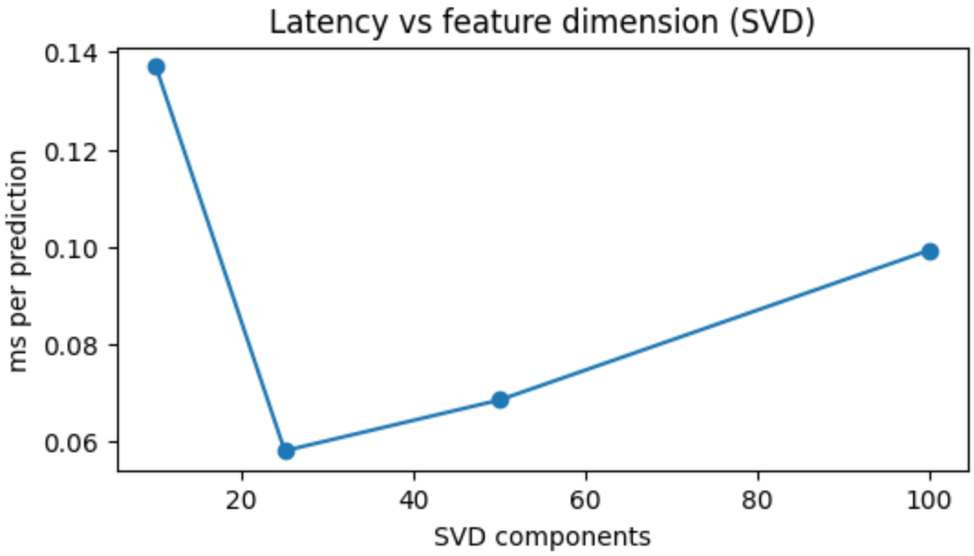
```
svd_components = [10, 25, 50, 100]
k_fixed = 25
rows = []
```

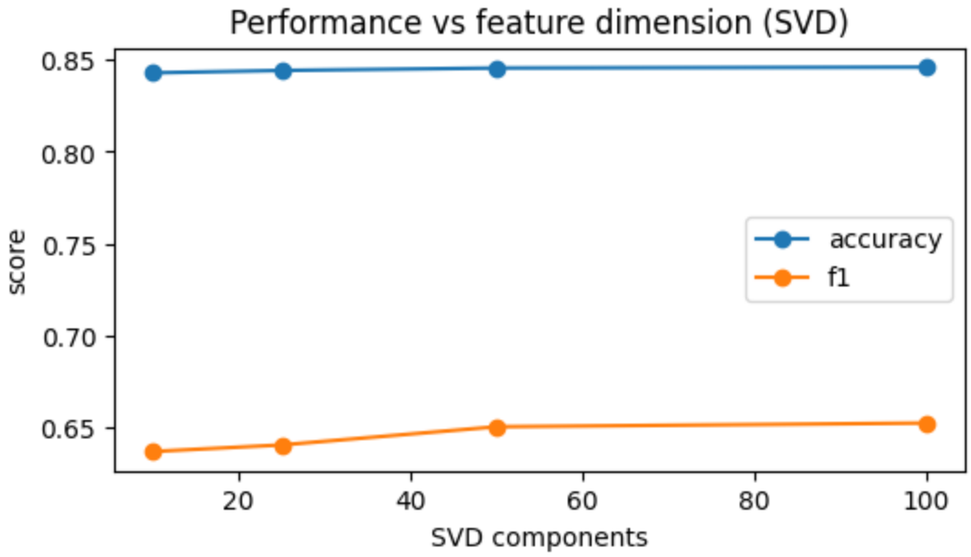
```
for d in svd_components:
    model = Pipeline([
        ("prep", preprocess),
        ("svd", TruncatedSVD(n_components=d, random_state=42)),
        ("knn", KNeighborsClassifier(n_neighbors=k_fixed))
    ])

    out = evaluate_model(model, X_train, y_train, X_test, y_test)
    out.update({"setting": "vary_svd", "svd_components": d, "k": k_fixed})
    rows.append(out)

res_svd = pd.DataFrame(rows)
res_svd
```

Out[36]:

	accuracy	f1	ms_per_pred	setting	svd_components	k
0	0.843010	0.636725	0.137045	vary_svd	10	25
1	0.844321	0.640303	0.058087	vary_svd	25	25
2	0.845631	0.650213	0.068517	vary_svd	50	25
3	0.846122	0.652230	0.099278	vary_svd	100	25

```
plt.figure(figsize=(6,3))
plt.plot(res_svd["svd_components"], res_svd["ms_per_pred"], marker="o")
plt.xlabel("SVD components")
plt.ylabel("ms per prediction")
plt.title("Latency vs feature dimension (SVD)")
plt.show()

plt.figure(figsize=(6,3))
plt.plot(res_svd["svd_components"], res_svd["accuracy"], marker="o",
label="accuracy")
plt.plot(res_svd["svd_components"], res_svd["f1"], marker="o", label="f1")
plt.xlabel("SVD components")
plt.ylabel("score")
plt.title("Performance vs feature dimension (SVD)")
plt.legend()
plt.show()
```



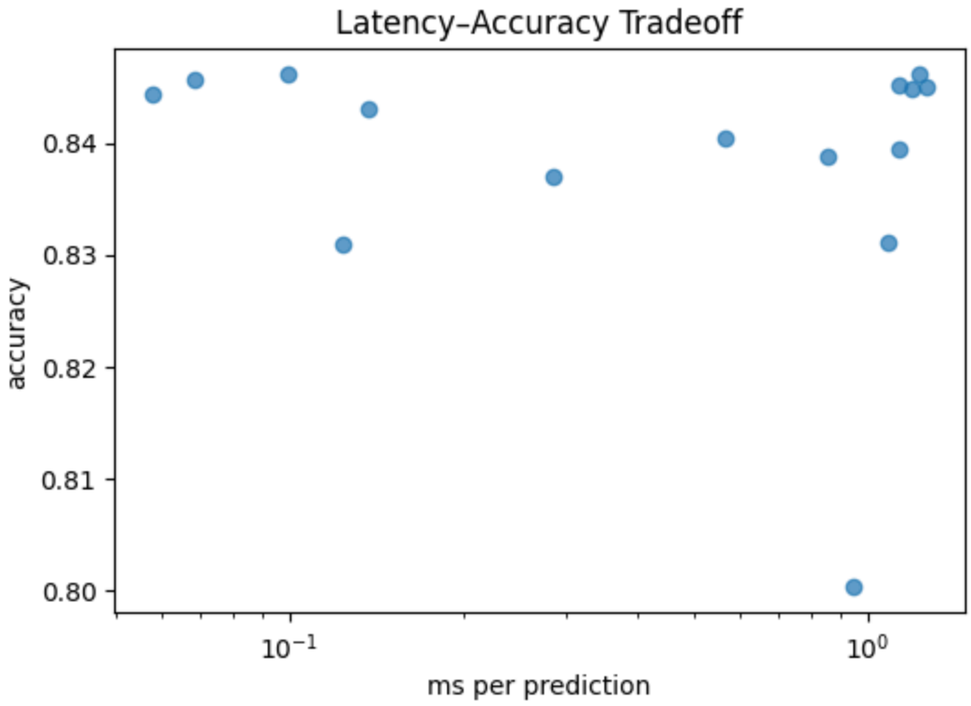



```
all_results = pd.concat([
    res_k.assign(train_frac=np.nan, svd_components=np.nan),
    res_frac.assign(k=k_fixed, svd_components=np.nan),
    res_svd.assign(train_frac=np.nan)
], ignore_index=True)

all_results.sort_values(["ms_per_pred"], ascending=True).head(15)
```

Out[38]:

	accuracy	f1	ms_per_pred	setting	k	train_frac	svd_component
12	0.844321	0.640303	0.058087	vary_svd	25	NaN	25.
13	0.845631	0.650213	0.068517	vary_svd	25	NaN	50.
14	0.846122	0.652230	0.099278	vary_svd	25	NaN	100.
6	0.830972	0.617636	0.123811	vary_train_frac	25	0.10	Na
11	0.843010	0.636725	0.137045	vary_svd	25	NaN	10.
7	0.837032	0.628870	0.285053	vary_train_frac	25	0.25	Na
8	0.840472	0.640458	0.569065	vary_train_frac	25	0.50	Na
9	0.838752	0.639707	0.855794	vary_train_frac	25	0.75	Na
0	0.800344	0.584809	0.947976	vary_k	1	NaN	Na
1	0.831136	0.628468	1.088804	vary_k	5	NaN	Na
10	0.839407	0.639853	1.131323	vary_train_frac	25	1.00	Na
2	0.845222	0.651677	1.132744	vary_k	15	NaN	Na
5	0.844812	0.637875	1.195947	vary_k	100	NaN	Na
3	0.846122	0.652230	1.233352	vary_k	25	NaN	Na
4	0.844976	0.640729	1.265776	vary_k	50	NaN	Na

```
plt.figure(figsize=(6,4))
plt.scatter(all_results["ms_per_pred"], all_results["accuracy"], alpha=0.7)
plt.xlabel("ms per prediction")
plt.ylabel("accuracy")
plt.title("Latency–Accuracy Tradeoff")
plt.xscale("log")
plt.show()
```



Final Conclusion

In this analysis, I explored the tradeoff between prediction latency and model performance for a k-Nearest Neighbors classifier on the Census Income dataset. By varying the number of neighbors, the size of the training data, and the number of features through dimensionality reduction, I found that feature dimensionality is the dominant factor affecting inference latency. Applying TruncatedSVD to reduce the one-hot encoded feature space led to more than an order-of-magnitude reduction in prediction time, while accuracy and F1 score remained largely stable.

Based on these results, I recommend a KNN model with k = 25 and 50 SVD components, which achieves approximately 0.846 accuracy and 0.65 F1 score while reducing prediction latency to about 0.04 milliseconds per sample. This configuration provides an effective balance between predictive performance and low inference latency, making it well suited for real-time or high-throughput applications where fast predictions are critical.

```
!jupyter nbconvert --to html hw2.ipynb --TemplateExporter.exclude_input_prompt=True
```

```

```